

Crash Avoidance for Autonomous Cars via Probabilistic Logic Counterfactual Reasoning

Héctor Avilés¹, Marco Negrete², Rafael Kiesel³, Alberto Reyes⁴

¹*Polytechnic University of Victoria*

²*School of Engineering - National Autonomous University of Mexico*

³*Vienna University of Technology*

⁴*National Institute of Electricity and Clean Energy*

Verónica Rodríguez⁵, Rubén Machucho⁶, Gabriel Ramírez⁷, Myriam Pequeño⁸

⁵*Technological University of the Mixteca*

^{6,8}*Polytechnic University of Victoria*

⁷*Center for Research and Advanced Studies*

Ingridh Gracia⁹, Nicolás Luévano¹⁰, Karelly Rivera¹¹, José Jesús Medrano¹²

^{9,10,11,12}*Polytechnic University of Victoria*

CONTENTS

22.1	Introduction	405
22.2	Related work	406
22.3	Methodology	407
22.3.1	Testbed and datasets	407
22.3.2	Learning causal Bayesian networks	410
22.3.3	Modeling and querying probabilistic logic twin networks	411
22.4	Evaluation and results	416
22.4.1	Test 1	416
22.4.1.1	Results and analysis of Test 1	417
22.4.2	Test 2	419
22.4.2.1	Results and analysis of Test 2	421
22.5	Conclusions and future work	423

We propose counterfactual reasoning through probabilistic logic twin networks (*PLTNs*) to avoid collisions in self-driving cars. *PLTNs* are grounded in causal Bayesian networks (*CBNs*) learned from expert knowledge, and simulated and synthetic data. *CBNs* incorporate the lane of the self-driving car, the presence of surrounding vehicles, an indicator for potential collisions, and six driving

actions. Counterfactual queries using PLTNs test different actions and select the one that minimizes the probability of a collision. We conducted two main tests for evaluation: (i) testing PLTNs trained with varying sample sizes and evaluated on random state-action examples known to lead to crashes, and (ii) deploying a simulated self-driving car equipped with a multi-layer perceptron (*MLP*) for behavior selection and extended with a counterfactual-based collision-avoidance subsystem, in three challenging driving scenarios. This vehicle is compared in the same scenarios with an autonomous vehicle built only on the MLP. Results demonstrate that counterfactual actions effectively reduce collision rates, highlighting their potential for developing collision avoidance strategies in self-driving cars. This is the first application of PLTNs in decision-making for autonomous vehicles.

22.1 Introduction

Collision avoidance is an important concern in self-driving cars to achieve full autonomous operation [581]. Although diverse approaches exist to accomplish this goal, many of them primarily rely on adjusting the current trajectory or behavior of the self-driving car to a new, safer course of action [582, 583, 584]. In this regard, we propose probabilistic counterfactual queries to explore *prospective*, hypothetical questions like:

“What is the probability that a potential collision could occur, given the current action and the state of the system, if a different action were chosen?”

Answers to that kind of “*What-if*” questions can help self-driving cars evaluate the convenience of alternative driving actions (or behaviors) to perform safer maneuvers under hazardous situations [585]. Additionally, the probabilistic causal models associated with causal questions can be framed in probabilistic logic programming [586], a paradigm well-suited for modeling causal relationships due to both its clear and flexible rule-based representation, and the availability of sophisticated inference procedures to efficiently solve probabilistic queries on complex probability distributions [587, 588].

Therefore, in this paper, we present our ongoing work on instrumenting counterfactual reasoning via probabilistic logic twin networks (*PLTNs*) to identify alternative actions that may prevent collisions in self-driving cars. PLTNs are twin networks [589] encoded as probabilistic logic programs. Inference is carried out through **Counterfactuals** [590], an efficient and effective software library recently developed to solve probabilistic counterfactual queries in PLTNs. Each PLTN is composed of two twin causal Bayesian networks (*CBNs*). A causal Bayesian network is a Bayesian network in which associations between variables are explicitly causal, not only statistical [591]. In this work, CBNs are constructed from expert knowledge, a large dataset of

examples obtained from multiple simulation runs of our self-driving car under both autonomous and human driving, and synthetic state-action examples. As state variables, the CBNs incorporate the lane in which the self-driving car is traveling and six “occupancy” variables that indicate the presence of up to four surrounding vehicles. These models also include six actions the self-driving car can choose in each state. When a potential collision is detected, the goal is to calculate the probability of the collision given the current state and action by intervening in the causal model with alternative actions, one at a time. It is assumed that the counterfactual action that minimizes the probability of a collision is the best alternative for the self-driving car to avoid or mitigate the severity of the impact. To evaluate our approach, we conducted two main tests. In the first test, five CBNs are constructed using 1%, 25%, 50%, 75% and 100% of a training dataset containing nearly 2 million state-action pairs. Each CBN is used to construct a single PLTNs model to compare results under varying training sample sizes. One hundred and twenty distinct examples were randomly selected from a pool of 288 unique state-action pairs identified as potential crashes in the database. These examples were independently queried to the five PLTNs to obtain the safest counterfactual action on each case. In the second test, a counterfactual-based collision-avoidance subsystem is incorporated into a simulated self-driving car equipped with a multi-layer perceptron (*MLP*) [592], which is a powerful predictive neural network applied here for behavior (or action) selection. The self-driving car was deployed in three highly difficult driving scenarios that almost certainly result in car collisions. This vehicle is compared with another simulated self-driving car built primarily on the MLP. Results from both tests show a noticeable decrease in car collisions when using probabilistic logic counterfactual reasoning. This finding suggests the viability of the proposed approach for developing a collision-avoidance module in self-driving cars.

The contribution of this paper is twofold. First, it presents the first application of probabilistic logic models for counterfactual reasoning in decision-making for autonomous vehicles. PLTNs integrate both symbolic reasoning and statistical learning into decision-making, enabling both prediction of possible outcomes and traceability of results. Additionally, PLTNs allow for explicit and clear expert knowledge representation, which facilitates model debugging and team collaboration, and also generates causality-based explanations that are understandable to humans. All of these are desirable features in critical decision-making applications such as autonomous driving. Second, unlike previous counterfactual approaches that implement retrospective analysis [593] or offline, pre-deployment policy evaluation [594], we explore a far less common prospective interpretation of counterfactual probabilities (that is, intervening with hypothetical actions to reason about safer alternatives before execution, during operational driving).

22.2 Related work

Studies in autonomous vehicles (*AVs*) have employed counterfactual simulations to evaluate the performance of *AVs* in preventing collisions and improving safety. For example, research on the Waymo Driver [595] demonstrated its effectiveness in avoiding fatal collisions by simulating crash scenarios and showing that it could prevent or mitigate a substantial percentage of crashes. Similarly, the impact of Advanced Driver Assistance Systems (*ADAS*) has been analyzed using counterfactual reasoning to assess their safety benefits and drawbacks in real-world scenarios [596]. This approach allows for a nuanced understanding of how different safety technologies might alter crash outcomes and driver behavior. Another relevant study focused on predicting crash configurations and the impact of Autonomous Emergency Braking (*AEB*) systems [597]. This research utilized counterfactual simulations to identify specific crash scenarios that *AEB* systems could not address, thus highlighting areas for improvement. Additionally, counterfactual reasoning has been applied to estimate the importance of objects in autonomous driving environments [598], enhancing the system's ability to prioritize critical objects and reduce collision risks. The effectiveness of pre-crash safety technologies, such as *AEB* and Electronic Stability Control (*ESC*), has also been evaluated in the context of reducing severe injuries in crashes [599]. This study underscores the role of these technologies in mitigating collisions and emphasizes the need for continued advancements to address remaining safety concerns. Furthermore, research on the design and evaluation of Automated Driving Systems (*ADS*) has utilized injury risk modeling to understand the potential outcomes of various crash scenarios [600]. By developing comprehensive injury risk surfaces, this research provides valuable insights into how different *ADS* designs could impact overall safety. The importance of considering driver behavior models in counterfactual simulations has been highlighted as well in [601]. Different models can significantly influence the effectiveness estimates of safety systems, underscoring the need for accurate simulations to evaluate safety technologies properly.

22.3 Methodology

In this section, we present the testbed for our simulated self-driving car, the databases constructed for evaluation, the learning process of CBNs, and the construction and querying procedure for PLTNs.

**FIGURE 22.1**

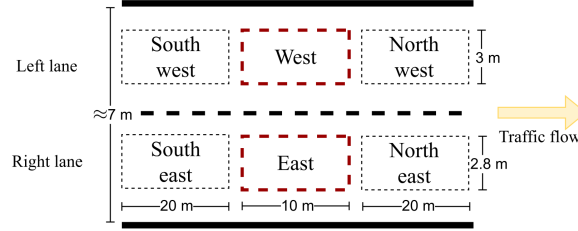
Race-like environment considered in this study for our self-driving car (in bright red).

22.3.1 Testbed and datasets

Our development framework involves a self-driving car simulated in race-like environments using the Webots 2022a simulator [602] and the Robot Operating System (*ROS*) Noetic [603] on Ubuntu 20.04. An example of the self-driving car and its environment is shown in Fig. 22.1. The self-driving car can travel on a two-lane road with straight segments and curves and up to 10 obstacle vehicles distributed over the road and either static or in motion. However, in this work, we are considering straight roads only. The task of the self-driving car is to pass all other vehicles. The sole task of the obstacle vehicles is to move straight at a constant speed. It is assumed that the vehicles traveling on the left lane moves faster than those on the right lane, while the self-driving car is the fastest (the maximum tested speed is near 50 km/h).

The original architecture of the self-driving car includes modules for *perception*, *driving control* and *decision-making*. Perception utilizes commercial sensor devices available in Webots. This module includes an accelerometer for collision detection, 3D laser readings for detecting other vehicles, and an RGB camera to detect lane borders. The control module is responsible for lane tracking, and speed and steering estimation, following standard control laws. The decision-making module selects driving behaviors based on two primary action policies estimated through probabilistic logic factored Markov decision processes (*PL-fMDPs*) [604, 605]. These policies promote traveling on the right lane while using the left lane only for overtaking. The vehicle behind the self-driving car in the same lane is not explicitly considered for behavior selection.

State variables are `curr_lane` which identifies the current lane of the self-driving car, and occupancy variables called `free_E`, `free_NE`, `free_NW`, `free_SE`, `free_SW` and `free_W` which indicate whether there is a vehicle or not in the location indicated in the name of each variable, relative to the self-driving car. Figure 22.2 depicts the locations represented by the occupancy variables on each lane. All these variables are Boolean. Finally, a multi-valued variable called `action` identifies one of four driving maneuvers for the self-driving car: `change_to_left` and `change_to_right` used for changing to the left lane and changing to the right lane, respectively, `cruise` to reach a

**FIGURE 22.2**

Predefined locations for other vehicles around the self-driving car (dashed red lines indicate the space the self-driving car occupy on each lane). When the vehicle is on the left (resp. right) lane, only the locations Northwest, Northeast, East and Southeast (resp. Northeast, Northwest, West and Southwest) are meaningful.

steady (maximum) speed, and **keep** to maintain a safe, steady distance to a vehicle ahead in the same lane.

In this work, we incorporate three key enhancements: (a) a new collision detection module based on a radar sensor for detecting obstacles in front of the self-driving car as well as estimating their relative velocity, (b) a variable called **latent_collision** used to warn potential lateral or frontal collisions according to the laser and radar readings, respectively, and (c) actions **swerve_right** and **swerve_left** which perform a controlled veer to the side of the lane while reducing speed. These evasive maneuvers are reserved for potential collision scenarios. Together with **keep**, they are considered the safest actions. Furthermore, to allow obstacle avoidance, the lanes have been widened from 3.5 m to approximately 5.3 m, providing enough empty space for swerving to the sides.

We have recorded several runs of the self-driving car system under both autonomous and human driving via a human-friendly interface developed on purpose¹. The combined size of these datasets is 1,238,869 examples of driving decisions collected over more than 600 km. The source code of our self-driving car and some videos of the system running are available at: <https://github.com/mnegrteev/WhatIf-Results/>.

Our approach requires labeling each state-action pair in the dataset as either leading to a collision or not. In the proposed driving environment, we consider that a crash may occur principally in two situations: (a) when the self-driving car applies the action **cruise** and there is a vehicle ahead, close enough in the same lane and possibly traveling in the same or opposite direction (resulting in a *rear-end* or *head-on* collision, respectively), and (b) when there is a car either next to or ahead in the lane the self-driving car merges into (*sideswipe* and rear-end crash, respectively). Following the previous cri-

¹These datasets are available at: <https://www.kaggle.com/autonomousvehicle/>

TABLE 22.1

Total number of state-action pairs labeled as potential and non-potential collisions in the integrated dataset, categorized by action.

Action	# of non-potential collisions	# of potential collisions	Total:
change_to_left	40,308	289,809	330,117
change_to_right	39,814	294,459	334,273
cruise	617,930	175,429	793,359
keep	496,558	0	496,558
swerve_left	2,883	0	2,883
swerve_right	1,681	0	1,681
Total:	1,199,174	759,695	1,958,869

teria, from the complete space of $2^7 \times 6 = 768$ state-action pairs, we carefully selected and labeled by hand 288 pairs as leading to a potential collision. Some state-action pairs that involve actions **change_to_left**, **change_to_right** and **cruise** emerged as “unsafe” driving pairs. With the new list of “safe” and unsafe state-action pairs as reference, we found that the automated and human control databases contain only a small number of potential crash examples (39,695 state-action pairs, all of which are repetitions of 132 unique pairs). Unfortunately, as of the time of writing, sampling potential crashes by human control has not been completed due to its tedious and time-consuming nature. Instead, the 288 unique state-action pairs were replicated 2,500 times to generate a synthetic dataset of 720,000 potential crashes. The number of repetitions was selected arbitrarily, with the aim of approximating a balance between collision and non-collision examples. During the process, continuous variables such as the speed and steering of the self-driving car were random sampled from their known data distributions, although they are not considered in the present work. This new dataset of crashing examples is integrated into the driving records originally obtained from the autonomous and human-controlled driving of the self-driving car. Let us call this integrated dataset $\mathcal{I}$. Dataset $\mathcal{I}$ is complemented by labeling each example as a potential collision or not using variable `latent_collision`.

Table 22.1 summarizes the total number of state-action pairs labeled as potential collisions or non-potential collisions in the integrated dataset, arranged by action. Table 22.2 shows the number of unique state-action pairs in the integrated dataset, grouped by action.

TABLE 22.2

Number of unique state-action pairs labeled as non-potential collisions and potential collisions (“safe” and “unsafe”, respectively) in the integrated dataset, grouped by action.

Action	# of unique “safe” pairs	# of unique “unsafe” pairs	Total:
change_to_left	14	112	126
change_to_right	15	112	127
cruise	53	64	117
keep	67	0	67
swerve_left	36	0	36
swerve_right	29	0	29
Total:	214	288	502

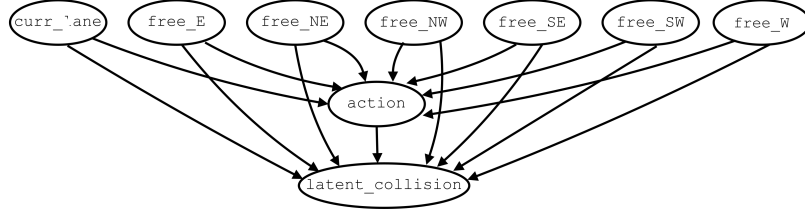
22.3.2 Learning causal Bayesian networks

Structural and parameter learning of CBNs were instrumented with the **bnlearn** package [606] in R [607]. Its *directed acyclic graph* is shown in Fig. 22.3. Structural learning was carried out by means of hill-climbing greedy search with Bayesian information criterion [608]. Initially, we devised a preliminary graph that we wished to test, so undesired links between variables were forbidden (for instance, those between occupancy variables, or those forming cycles, for example, a link from **action** to **curr_lane**). In this manner, hill-climbing decides which of all the permissible connections should be kept or removed. The resulting structure resembles a causal graph with confounders, where **action** is the treatment variable, **latent_collision** is the output variable and the current lane and occupancy variables collectively act as a multi-valued confounder.

The parameters of the CBN were fitted by the *maximum likelihood estimation* criterion [609]. Let

state = (**curr_lane**, **free_E**, **free_NE**, **free_NW**, **free_SE**, **free_SW**, **free_W**)

be the tuple that indicates the state of the environment. Training data is organized into (**state**, **action**, **latent_collision**) triplets. For training, we considered the integrated dataset containing 1,958,869 examples. From our perspective, data leakage (which occurs when some samples are used for both training and testing) is not a major concern in this setup. This is because our proposal focuses on minimizing conditional probabilities under interventions in PLTNs, rather than assessing their generalization capabilities for predicting outcomes in previously unseen data.

**FIGURE 22.3**

Causal Bayesian network proposed in this work.

22.3.3 Modeling and querying probabilistic logic twin networks

Counterfactuals package requires as its input a CBN extended with *exogenous*², *unobserved*, *error terms*. The CBN must be encoded as a probabilistic logic program written in ProbLog. We assume the reader has some familiarity with Prolog and ProbLog syntax. For an introduction we refer to [610] for Prolog and [611] for ProbLog. To achieve this input format, we developed a purpose-built R script to syntactically translate any CBN learned with **bnlearn** into a ProbLog program. Consider the probabilistic logic program P_{CBN} of a simple CBN described in Listing 22.1 and learned from the integrated dataset, as indicated in the previous section. This is an excerpt of the CBN presented in Figure 22.3. The new, smaller CBN, shown on the left side of Figure 22.4, omits all state variables except **free_NE**, which directly influences **action** and **latent_collision**. This model comprises only the actions **cruise** and **keep**.

Listing 22.1

ProbLog code for the causal Bayesian network of our simple example.

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Exogenous variable
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

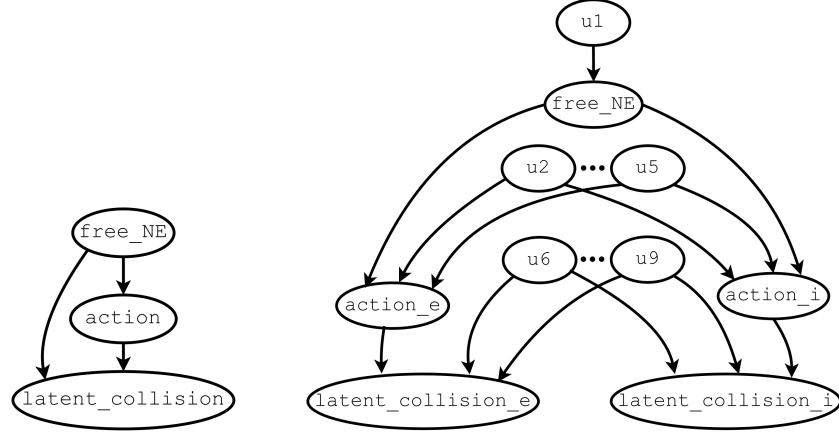
0.466::free_NE.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Rules
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

0.331::action(cruise) :- \+ free_NE.
0.668::action(keep) :- \+ free_NE.
0.939::action(cruise) :- free_NE.

```

²An exogenous variable is one that is not influenced by any other variable.

**FIGURE 22.4**

Example of a simple causal Bayesian Network (left) and its corresponding twin networks model (right).

```
0.061::action(keep) :- free_NE.

% Integrity constraint to enforce mutual exclusivity
% between action(cruise) and action(keep) (they cannot
% be both true at the same time)
:- action(cruise), action(keep).

0.576::latent_collision :- action(cruise), \+ free_NE.
0.001::latent_collision :- action(keep), \+ free_NE.
0.077::latent_collision :- action(cruise), free_NE.
0.001::latent_collision :- action(keep), free_NE.
```

Initially, to construct the corresponding PLTNs model, two copies of the CBN, including the structure and parameters of the *endogenous* variables³, are created. Endogenous variables are `free_NE`, `action` and `latent_collision`. The first copy represents the variables in the (factual), “evidence” world, while the second copy stands for the (hypothetical) “intervention” world. The right side of Figure 22.4 depicts the twin networks model. The suffix “_e” identifies endogenous variables in the evidence world, while the suffix “_i” identifies those in the intervention world. These two network copies are linked through error terms `u*`, which adhere to the **C**ounter**F**actuals notation for error terms, while also duplicating the original causal structure of the endogenous variables. The role of the error terms is to account for unobserved factors that introduce variability into the deterministic relationships (described by the logic rules) between causes and their effects. Moreover, they allow the effect of evidence

³An endogenous variable is one that is influenced by other variables.

to propagate into the hypothetical world, as described below. The resulting probabilistic logic twin networks model (referred here to as P_{TN}) is shown on the right side of Figure 22.4, and its ProbLog code is presented in Listing 22.2.

Listing 22.2

ProbLog code for the PLTNs of our simple example.

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Error terms
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

0.466::u1.
0.331::u2.
0.668::u3.
0.939::u4.
0.061::u5.
0.576::u6.
0.001::u7.
0.077::u8.
0.001::u9.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Factual network (evidence world)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

free_NE_e :- u1.

action_e(cruise) :- \+ free_NE_e, u2.
action_e(keep) :- \+ free_NE_e, u3.
action_e(cruise) :- free_NE_e, u4.
action_e(keep) :- free_NE_e, u5.

% Integrity constraints
:- action_e(cruise), action_e(keep).

latent_collision_e :- action_e(cruise), \+ free_NE_e, u6.
latent_collision_e :- action_e(keep), \+ free_NE_e, u7.
latent_collision_e :- action_e(cruise), free_NE_e, u8.
latent_collision_e :- action_e(keep), free_NE_e, u9.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Counterfactual network (intervention world)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

free_NE_i :- u1.

action_i(cruise) :- \+ free_NE_i, u2.
```

```

action_i(keep) :- \+ free_NE_i, u3.
action_i(cruise) :- free_NE_i, u4.
action_i(keep) :- free_NE_i, u5.

% Integrity constraints
:- action_i(cruise), action_i(keep).

latent_collision_i :- action_i(cruise), \+ free_NE_i, u6.
latent_collision_i :- action_i(keep), \+ free_NE_i, u7.
latent_collision_i :- action_i(cruise), free_NE_i, u8.
latent_collision_i :- action_i(keep), free_NE_i, u9.

```

Here, we want to query conditional probabilities of the type:

$$p(\text{latent_collision_i} | \text{latent_collision_e}, \text{free_NE_e}, \text{action_e}, \text{do}(\text{action_i})) \quad (22.1)$$

where $p(\cdot|\cdot)$ is a conditional probability function, $\text{latent_collision_e}$, free_NE_e and action_e are all evidence, and $\text{do}(\cdot)$ is the *do*-operator [591], which introduces an intervention over action_i . The objective is to identify a hypothetical action action_i , via the PLTNs, that minimizes the probability of $\text{latent_collision_i}$. Closely associated with twin networks are structural causal models (*SCMs*) [591]. The standard procedure for querying SCMs (which can also be applied to twin networks) involves three steps: (i) the *abduction* step which incorporates the evidence into the model and propagates its effect to the intervention world by accounting for the causal effect of the error terms, (ii) the *action* step to assert the intervened action in the intervention world, attenuating the effect of the evidence, and (iii) the *prediction* step to calculate the probability of the output variable in the intervention world (in our case, the probability of a possible collision) given the evidence and the intervention. Instead, **Counterfactuals** applies probabilistic logic programming which performs the next steps⁴:

1. Intervene the original PLTNs program P_{TN} by asserting the intervention action, which involves removing rules that derive the intervened atom or have bodies inconsistent with the intervention, and adding rules to introduce positive interventions. The resulting program is denoted P'_{TN} .
2. Identify and construct a program P_r with logical atoms relevant to the query and the evidence at hand (atoms associated with the evidence, the intervention and their ancestors) from P'_{TN} by using an and/or graph.

⁴**Counterfactuals** supports both single and multiple queries to different variables using the same evidence and intervention, though we focus on single queries as we are interested in a single variable per query.

3. Ground facts and rules in P_r by creating a *grounded program* P_g ⁵ (in our example, all atoms are already grounded).
4. Perform Clark's completion over P_g to handle *negation as failure* [612], producing a propositional formula, which is then converted to conjunctive normal form (*CNF*), denoted F_{CNF} ⁶.
5. Use **sharpsat-td** [613] to compile F_{CNF} into a *smooth deterministic decomposable negation normal form* (*sd-DNNF*) formula [614], denoted F_{sd} , using tree decomposition.
6. Solve the conditional probability query over F_{sd} via *weighted model counting* [615] leveraging the **aspmc** package [616].

For example, suppose it was observed that **latent_collision_e=T** (a potential collision has been warned⁷), **free_NE_e=F** (the nearby space ahead is occupied) and **action_e=cruise**. This means there is no empty space ahead, but the self-driving car is moving forward anyway. Additionally, assume **action_i=keep** (the counterfactual action for which the marginal probability of **latent_collision_i=T** will be calculated). The probability of **latent_collision_i=T** given the evidence and **keep** as the intervention is approximately 0.001, suggesting this action is an plausible alternative to avoid the crash.

22.4 Evaluation and results

This work conducts two tests to qualitatively and quantitatively assess counterfactuals in selecting hypothetical actions to prevent collisions, both offline and in simulation.

22.4.1 Test 1

To perform this test, five PLTNs are constructed and evaluated independently. First, a random sample of sixty⁸ (**state**, **action**, **latent_collision**) triplets where both **latent_collision = T** and the tuple **state** is unique are selected from the integrated dataset $\mathcal{I}$. We refer to this subset as $\mathcal{S}_{unique}$, which will

⁵A logic program is grounded when none of the arguments of any *atomic* propositions is a variable or when their arguments do not contain variables.

⁶PLTNs inherits acyclicity of CBNs, so cycle-breaking, commonly required to infer from some logic programs, is not necessary.

⁷**T** means the truth-value true, and **F** means false (in the case of the occupancy variables, **T** indicates that a location is empty and **F** that it is occupied by another vehicle).

⁸Note that the crash examples in the dataset $\mathcal{I}$ (i.e., those in which **latent_collision = T**) comprises all of the $2^7 = 128$ states. Therefore, we are selecting nearly half of the possible states for testing.

serve later as the basis for constructing the counterfactual queries for evaluation. Next, all examples in $\mathcal{I}$ matching any of the unique states in $\mathcal{S}_{unique}$ are removed. The remaining $\mathcal{I}$ subset, which has a size of 1,164,315 examples is reserved as the primary training dataset. This sample is denoted by $\mathcal{I}_{\mathcal{T}}$. By removing examples selected by **state**, we ensure testing data is unknown to the PLTN models, thereby eliminating the risk of data leakage and, at the same time, ensuring that all PLTNs are evaluated on the same test data. Five random samples of size 1%, 25%, 50%, 75%, and 100% from $\mathcal{I}_{\mathcal{T}}$, stratified by **action**, are extracted to learn the structure and parameters of five CBNs, as described in Section 22.3.2. Each CBN is then used to construct a PLTNs model.

Interventions are introduced into each example in $\mathcal{S}_{unique}$ by pairing the corresponding triplet with each of the six possible actions, producing six (**state**, **action**, **latent_collision**, **intervention**) quartets. These six quartets constitute a single, unique testing group, where **intervention** $\in \{\text{change_to_left, change_to_right, cruise, keep, swerve_right, swerve_left}\}$. In other words, each quartet in the group shares the same triplet (**state**, **action**, **latent_collision**), but incorporates an intervened action as new information to assess its effect on **latent_collision**. Therefore, 60 groups, each consisting of 6 quartets are created. By following suffixes introduced in Section 22.3.3 for intervention and evidence variables, the minimization of the probability of collision within each group can be formalized as:

$$\begin{aligned} \text{Actions}^* = \arg \min_{\text{action_i}} & p(\text{latent_collision_i} | \text{latent_collision_e}, \\ & \text{curr_lane_e}, \text{free_E_e}, \text{free_NE_e}, \text{free_NW_e}, \\ & \text{free_SE_e}, \text{free_SW_e}, \text{free_W_e}, \text{action_e}, \\ & do(\text{action_i})) \end{aligned} \quad (22.2)$$

where Actions^* denotes a set with one or more interventional actions **action_i** that minimize the probability of a potential collision, given the current evidence and counterfactual intervention. Multiple actions may belong to Actions^* if they all result in the same minimum probability value for **latent_collision_i**.

Each of the five PLTNs solves the same test set of $60 \times 6 = 360$ queries (i.e., individual test cases). Table 22.3 summarizes the number of unique state-action pairs used for learning and testing PLTNs for each driving action and training sample size.

22.4.1.1 Results and analysis of Test 1

This test subjectively evaluates the potential of action interventions to prevent collisions and analyzes the frequency of ties among actions.

To evaluate the correctness of the interpretation of the selected counterfactual actions as “safe”, we assume that each **action** $\in \text{Actions}^*$, along with

TABLE 22.3

Number of unique state-action pairs (stratified by action) used in the construction and testing of the five PLTNs.

Action	PLTNs model					Test dataset
	1%	25%	50%	75%	100%	
change_to_left	67	68	68	68	68	21
change_to_right	63	65	67	67	67	26
cruise	59	62	63	63	63	13
keep	22	39	42	43	47	0
swerve_left	4	17	21	23	23	0
swerve_right	5	15	17	17	18	0
Total:	220	266	278	281	286	60

TABLE 22.4

Number of actions suggested by PLTNs that are safe or unsafe actions, accordingly to our initial labeling in Test 1.

Action label	PLTNs model				
	1%	25%	50%	75%	100%
Safe	68	103	107	114	119
Unsafe	0	7	5	7	6
Total:	68	110	112	121	125

state, must match the initial labeling in dataset $\mathcal{I}$ for their corresponding (**state**, **action**, **latent_collision** = **T**) triplet. Table 22.4 summarizes the number of actions suggested by the PLTNs that are safe or unsafe according to our initial state action labeling. It is shown that the actions inducing the lowest probability of **latent_collision** often align with collision avoidance across all PLTNs models. Table 22.5 reports the total occurrences of each action minimizing collision probability across training percentages.

From these tables it can be seen that the number of actions suggested by PLTNs consistently exceeded the number of unique test groups (that is, 60), especially as the size of the training sample increased. Since one would typically expect a single best action per test group, this indicates that, in several groups, multiple actions were tied as the most appropriate intervention. Therefore, to further analyze ties, we rank the six quartets within each group by their **latent_collision_i** probability from lowest (best) to highest. Table 22.6 summarizes how many times the specified number of actions shared the

TABLE 22.5

Number of times each action was selected as the most appropriate intervention across all training percentages in Test 1.

Action	PLTNs model				
	1%	25%	50%	75%	100%
change_to_left	0	7	7	8	9
change_to_right	3	6	6	7	6
cruise	25	25	21	22	23
keep	26	36	37	38	39
swerve_left	7	21	24	29	29
swerve_right	7	15	17	17	19
Total:	68	110	112	121	125

first place (that is, they tied for the lowest predicted likelihood of a crash) across 60 test groups per training percentage. No six-way ties occurred.

Remarkably, from the last column of Table 22.6, 78.6% of the decisions made through all the models involve only one or two best actions tied for the first place, while 90% of the decisions involve between zero and three ties. Interestingly, the dispersion of ties appears to increase slightly as the size of the training set grows.

The increase in both the number and dispersion of ties may be attributed to several factors: noise in the training dataset such as inconsistent or misleading labels which affects counterfactual decisions, a model with limited expressiveness due an oversimplified structure, or more likely according to our prior knowledge, due to a bias introduced by highly repeated examples dominating the dataset. However, a thorough analysis of these factors is beyond the scope of this paper.

This test was conducted on a laptop with an Intel Core i7-3610QM (2.3GHz) processor and 8GB of RAM. The average time to solve each counterfactual query across the three models was 5.74 seconds (SD = 0.15). Despite the effectiveness of PLTNs in identifying safer actions, the average inference time per counterfactual query exposes a computational efficiency limitation for real-time use. This arises from the combinatorial complexity of probabilistic logic inference, sequential grounding in ProbLog, and absence of hardware-specific optimizations. Although the laptop used is not high-performance, dedicated hardware and inference optimizations [617], as well as precomputing prototypical queries, can improve runtime performance.

TABLE 22.6

Number of actions in the first place (that is, actions with the same minimum likelihood of a crash) across the six-intervention groups and training percentages in Test 1.

# of actions in 1 st place	PLTNs model					Total:
	1%	25%	50%	75%	100%	
1	53	29	30	27	25	164
2	6	18	16	15	17	72
3	1	8	8	10	8	35
4	0	4	4	6	8	22
5	0	1	2	2	2	7
Total:	60	60	60	60	60	300

22.4.2 Test 2

This test evaluates a simulated self-driving car using an MLP for behavior selection and a counterfactual-based collision-avoidance subsystem⁹. The MLP has three hidden layers (eight nodes each) with ReLU activation, and is optimized via Adam with 500 iterations. Both the MLP and the CBN for collision avoidance were trained on a stratified 90% sample of the dataset $\mathcal{I}$. Real-time counterfactual probability computation was infeasible, so probabilities for all 768 (**state**, **action**) pairs labeled as potential collisions were precomputed and stored in a CSV file for consultation during deployment¹⁰. Ties for the first place between interventions were manually resolved by one of the authors, followed by group discussion. This manual selection was applied based on subjective and conservative criteria, prioritizing safety or intuitive maneuvers according to our judgment. Table 22.7 depicts the number of times each action was selected. The number of queries in which we must decide which action to choose for each lane is $\sum_{k=1}^5 C(6, k) = 63$, where $C(6, k)$ denotes the number of possible combinations of k tied actions from a set of 6 available actions. Therefore, the total number of manual decisions across both lanes is $2 \times 63 = 126$. In general, we prefer cruising over changing lanes, swerving, and keeping distance, and prefer changing lanes over swerving and keeping distance. Manual resolution of ties between interventions through expert-driven selection, as implemented in this study reflects an interim approach to decision-making, which may introduce the possibility of anthropic bias, and therefore, it may not generalize well across all contexts or evaluators.

⁹Training the MLP involves (**state**, **action**) pairs, while training a CBN requires (**state**, **action**, **latent_collision**) triplets.

¹⁰We assume any state-action pair initially cataloged as safe may become unsafe when facing unexpected driving scenarios or differences between the primary perception module and the newly implemented collision detection module

TABLE 22.7

Number of times each action was manually selected by the authors for tie-breaking in Test 2.

Action	# of times selected
cruise	64
change_to_left	18
change_to_right	16
swerve_left	12
swerve_right	11
keep	5
Total:	126

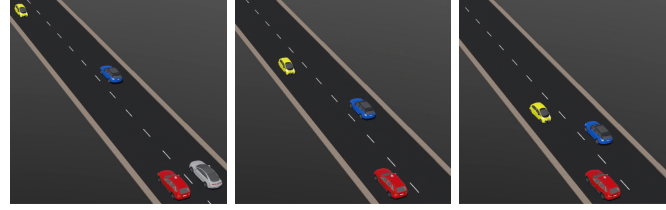
The vehicle was tested in three complex, high-risk driving scenarios distinct from training. The environment consists of a straight, two-lane road with up to three obstacle vehicles, requiring the self-driving car to pass without crashing. The scenarios evaluated in this paper make use of certain simplifications of the autonomous driving problem, such as fixed lane widths, deterministic behavior of other vehicles, and the use of synthetic data. These assumptions allowed for the analysis of counterfactual reasoning under controlled conditions.

The three scenarios are: (a) overtaking on the left while an oncoming vehicle approaches (head-on crash), (b) overtaking in the right lane while two vehicles approach rapidly from behind in the left lane (sideswipe collision), and (c) passing through a crossroads while a vehicle fails to yield (T-bone accident). Figure 22.5 illustrates these scenarios. The MLP selects actions at ≈ 10 Hz, but when a potential collision is detected via radar or laser scans, the collision-avoidance subsystem overrides the MLP, continuously querying counterfactual actions until the risk subsides. In scenario (a), to prevent an inevitable crash due to the narrow lanes (≈ 5.3 m each), the yellow vehicle moving in the opposite direction is the only obstacle vehicle equipped with obstacle detection to decide when to veer to its left, in all tests. The maximum speed of the self-driving car is 30 km/h, while obstacle vehicles range from 10 km/h to 54 km/h. Initial vehicle positions were predetermined, with random forward/backward shifts of ± 2 m before each trial.

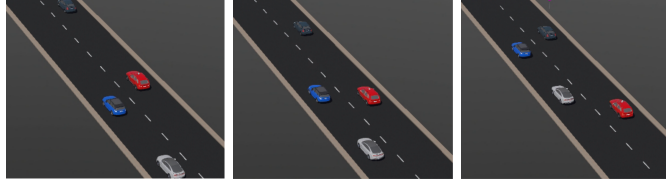
The MLP was implemented in Scikit-learn [618], and simulations ran on an AMD Ryzen 9 5900X (12-core, 24-thread) with 32GB RAM on Ubuntu 20.04.6 LTS.

22.4.2.1 Results and analysis of Test 2

The evaluation is based on the number of runs in which the self-driving car successfully evaded obstacle vehicles and continued on its path. Table 22.8 summarizes the number of successful runs for each scenario and decision mod-



(a) A vehicle (in yellow) approaching in opposite direction.



(b) Overtaking with fast vehicles approaching from behind on the left.



(c) Crossing an intersection with a reckless vehicle (in gray) approaching in transverse direction.

FIGURE 22.5

Challenging driving scenarios for our self-driving car (in bright red): (a) a vehicle (in yellow) approaching in the opposite direction in the left lane, (b) overtaking a vehicle (in dark gray) while fast-moving vehicles approach from behind in the left lane, and (c) crossing an intersection while a vehicle (in light gray) fails to yield the right-of-way.

ule, as well as the total number of successful runs per module. In scenarios (a) and (b), the self-driving car using a combination of MLP and counterfactual reasoning via PLTNs, clearly outperformed the vehicle with a decision-making module based solely on the MLP, but this outcome is not entirely surprising. It is well established in the literature that traditional supervised learning approaches may struggle to generalize to unseen testing conditions, particularly when faced with moderate to significant data distribution shifts resulting from changes in the data generation process. Augmenting the MLP-based decision strategy with counterfactual actions resulted in nearly perfect performance, achieving $\frac{90}{90} = 100\%$ of successful runs, compared to the MLP alone, which achieved only $\frac{4}{90} = 4.4\%$. This demonstrates counterfactuals improved success by a factor of 22.72 in the first two scenarios. The evasion behavior incorporated to the yellow vehicle in scenario (a) seems to potentially favor collision avoidance for both self-driving cars, in particular, allowing the MLP-based ve-

TABLE 22.8

Number of successful runs for the two decision-making modules assessed in Test 2, with 30 repetitions for each model in each driving scenario (90 in total for each model).

Scenario	# of runs per module	
	MLP+PLTNs	MLP only
(a) Approaching in opposite direction	30	4
(b) Overtaking with fast vehicles behind	30	0
(c) Crossing intersection	1	0
Successful runs by module:	61	4

hicle to complete some trials without collisions. However, we believe this type of cooperation represents a controlled condition justifiable through vehicle-to-vehicle communication, where autonomous vehicles coordinate themselves. Nevertheless, we recognize further evaluation is needed to emulate more realistic driving scenarios, allowing the yellow vehicle to swerve to either side, slow down or continue in the same direction. Finally, the low performance of both approaches in scenario (c) indicates that the current regions of interest (shown in Figure 22.2) scanned by the self-driving car are insufficient for detecting fast-approaching vehicles from a transverse direction, which, in turn, could require refinements to the causal model.

The scenarios evaluated in this paper make use of certain simplifications of the autonomous driving problem, such as fixed lane widths, deterministic behavior of other vehicles, and the use of synthetic data. These assumptions allowed for the analysis of counterfactual reasoning under controlled conditions. As future work, extensions toward more realistic environments and vehicles with stochastic behavior will be considered.

22.5 Conclusions and future work

This work establishes probabilistic logic twin networks (PLTNs) as a robust framework for counterfactual reasoning in autonomous vehicle decision-making, leveraging causal Bayesian networks constructed from expert knowledge, simulated environments, and synthetic data. PLTNs enable systematic evaluation of “what-if” scenarios to identify actions that minimize collision probability, while enhancing the transparency and traceability of decision processes. Experimental results demonstrate that integrating PLTNs with multi-layer perceptron (MLP) modules yields a substantial reduction in collision

rates compared to MLP-only baselines, especially in challenging driving scenarios. The approach allows for alternative maneuvers that override current actions when potential collisions are detected, and probabilistic logic further enhances explainability by tracking inference paths and clearly describing complex spatial relationships between vehicles. These findings highlight the potential of PLTNs as reliable and interpretable tools for collision avoidance in autonomous driving.

Future research will focus on overcoming current limitations to achieve real-time performance by developing hardware-accelerated inference engines and advanced knowledge compilation techniques. Additionally, we will implement automated, multi-criteria decision-making protocols to address the subjectivity inherent in manual tie-breaking. Moreover, we will expand the causal model to incorporate discrete distance and direction variables, as well as evaluate the approach in dynamic, stochastic environments with enhanced sensor arrays, will further improve the applicability of the system on more realistic driving scenarios. Finally, we will carry out systematic comparisons with other existing collision avoidance approaches. These advancements are expected to solidify probabilistic logic and counterfactual reasoning as foundational tools for reliable, transparent collision avoidance in real-world autonomous driving.

Acknowledgments. This work was partially founded by UNAM-DGAPA under grant IT102424 and AI Consortium - CIMAT-CONAHCYT.